

Java Script

a

API's: Application Programming Interface, interfaz que permite que dos sistemas o programas se comuniquen entre sí.

En JavaScript, usamos APIs para pedir datos externos, como el clima, usuarios, productos, etc.

Ej. `fetch("https://api.clima.com/data")` // usamos una API para obtener datos del clima

Array: es tipo de dato, matrices unidimensionales o arreglos, listado/ conexión valores

[] corchetes definen un arreglo

Longitud: const cantidad de valores dentro de los corchetes, separados por una coma , () es dinámica; se pueden agregar mas elementos.

Índice / posición: El primer elemento tiene la posición 0 y así sucesivamente

Ej. `console.log(numerosPares[posición]);`

Palabra reservada: **const**

Como definir un arreglo

Forma 1: declarar y asignar valores desde el principio (const no impide agregar mas elementos)

```
let colores = [];  
// arreglo vacio
```

```
let colores = ["rojo", "verde", "azul", "morado", "amarillo"];
```

```
// base arreglo
```

```
console.log(colores);
```

```
// la consola muestra longitud, con la flecha desplegable muestra la posición
```

Forma 2: declarar arreglo y después asignar valores

```
let listaCompras = [];
```

```
listaCompras[0] = "Leche";
```

```
// por default usara el cero
```

```
listaCompras[1] = "Huevos";
```

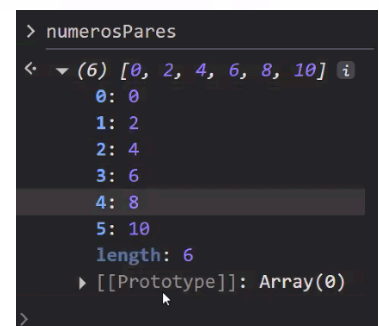
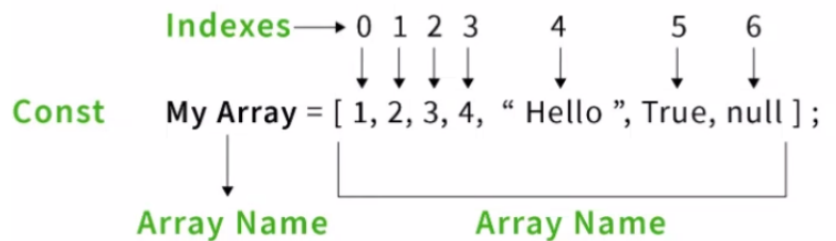
```
listaCompras[2] = "Queso";
```

```
console.log(listaCompras);
```

Forma 3: utilizando un constructor Array

```
let numerosImpares = new Array(1, 3, 5, 7, 9, 11);
```

```
console.log(numerosImpares);
```



Java Script

Métodos: función previa dentro del lenguaje
Inicia con **.** + **nombre** + (**parámetro**)

Metodos para agregar elementos a arrays

.push("elemento")

agrega un elemento al final

```
colores.push("rosa");  
console.log(colores);
```

.unshift("elemento")

agrega un elemento al inicio

```
colores.unshift("naranja");  
console.log(colores);
```

Elimina un elemento

.pop() elimina el elemento al final

```
colores.pop();
```

.shift() elimina el elemento al inicio

```
colores.shift();
```

Modifica

Metodo iterativo

.map(parámetro=>función=): itera sobre todos los elemento devolviendo un nuevo arreglo

```
const funciónFlecha = (parámetro) =>  
instrucciones
```

```
let numerosImpares = new Array(1,  
3, 5, 7, 9, 11);
```

Darle nombre a todos los parámetros del arreglo

```
console.log(numerosImpares);
```

```
let numerosPares = numerosImpares.map(function (numerosImpares){  
// funciones anonimas, no se les asignan un nombre  
  return numerosImpares -1;  
});
```

```
console.log(numerosPares);
```

Con función flecha

```
let numerosPares = numerosImpares.map(numerosImpares => numerosImpares -1)
```

.forEach(): itera sobre todos los elementos de un arreglo

```
colores.forEach(function (color){  
  console.log(color)  
});
```

```
colores.forEach(color => console.log(color.toUpperCase()));
```

JAVASCRIPT ARRAY METHODS CHEATSHEET



Java Script

b

Boolean: datos binarios true/false

Uso de estructuras de control; ampersand & and && | OR ||

```
true && true; true
false && true; false
true && false; false
false && false; false
```

```
true || true; true
false || true; true
true || false; true
false || false; false
```

Bucle - ciclo - iteración: Otro tipo de estructura de control, depende de una condición.

Se repite el procesos hasta que finalmente se deje de cumplir una condición.

También se usa para recorrer elementos

For

```
let numerosPares = [0, 3, 4, 6, 8, 10];

for (inicio; condición; contador){
  instrucciones
}

// ciclo que imprima los numeros del 0 al 15
for(let i = 0; condición; contador){
  // i puede llamarse como sea
  Instrucciones
}

for(let i = 0; i <= 5; i = i++){
  // i puede llamarse como sea
  // i++ es igual a i+1
  console.log(i);
}
```

While

```
while (condition){
  instrucciones
}

let i = 1

while(i <=10){
  i++;
  console.log(n);
}
```

Java Script

Diferencia: `for` es estructura fija, `while` estructura flexible (contador) y `do while` ejecuta al menos 1 vez 1 acción.

Do while primero ejecuta una acción antes de evaluar condición

```
do{  
  
}while ()  
  
Ej.  
let x = 0;  
do{  
    console.log(x);  
    x++;  
}while (x<=1)
```

c

Constantes: variable cuyo valor no puede cambiarse una vez asignado.

Palabra reservada: **const**

Convención: UPPER_SNAKE_CASE

Ej. `const PI = 3.14;`

Error en consola por temas de scope: `TypeError: Assignment to constant variable.`

Concatenar datos:

```
let name = "Erika";  
let mail = "eahg@mail.com";  
let mensaje = "My name is " + name + " my mail is " + mail;
```

Uso de backtics ``

```
let mensaje = `My name is ${name} n my mail is ${email}`;  
My name is Erika\my mail... \ significa salto de linea
```

Constructores: función/método especial que se usa para crear objetos desde la clase.

Sirve como molde para generar múltiples objetos con las mismas propiedades o métodos.

Se usa una función con mayúscula inicial, y se invoca con **new**.

this: palabra reservada, se refiere al objeto que se está creando.

new: crea un objeto vacío y le asigna el prototipo correcto.

Se usa para programar de forma orientada a objetos antes de class (ES6).

```
Ej. function Persona(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
}
```

```
let erika = new Persona("Erika", 22);  
console.log(erika.nombre); // "Erika"
```

Java Script

d

Datos: define qué **clase de valor** puede almacenar una variable, y qué **operaciones** se pueden hacer con ese valor.

Primitivos: valores simples, inmutables, no se puede cambiar directamente su contenido, solo reemplazar el valor. Se guardan por valor. No tienen métodos ni propiedades propias (aunque pueden comportarse como objetos de forma temporal por el motor de JS)

Tipo	Características	Ejemplo
String	Texto o caracteres entre comillas	"Hola"
Number	Números (enteros o decimales)	10, 3.14, -5
Boolean	Datos lógicos: Verdadero o falso	true, false
Undefined	Valor que no ha sido asignado	let x; ⇒ x === undefined
Null	Ausencia intencional de valor	null
BigInt	Números enteros grandes (más allá del límite de Number)	12345678901234567890n
Symbol	Identificadores únicos (usado poco fuera de proyectos grandes)	Symbol("id")

Complejos (de referencia)/composite/non-primitive: estructuras más complejas, que se almacenan por referencia, no por valor, es decir si al hacer una copia de ellos, ambas variables apuntan al mismo objeto en memoria, pueden almacenar múltiples valores y estructuras. Son mutables y se accede a ellos por referencia, no por valor. (Son objetos en el fondo)

Tipo	Características	Ejemplo
Object	Agrupar pares clave-valor	{ nombre: "Erika", edad: 22 }
Array/arreglo	Lista ordenada de valores	[1, 2, 3]
Function	Bloques de código reutilizable	function saludar() {}
Date, Map, Set	Más estructuras complejas	Nuevos tipos derivados

Conversion de tipos

Conversion	Ejemplo
Implícita	"5" + 2 // "52" (concatenación) "5" - 2 // 3 (conversión a number) true + 1 // 2 (true se convierte a 1) false + 1 // 1

Java Script

Explícita	Number("5") // 5 String(123) // "123" Boolean(0) // false Boolean("hola") // true
-----------	--

Comparación

Comparación	Características
"5" == 5 // true null == undefined // true	Compara valores pero permite conversión de tipo
"5" === 5 // false ✗ null === undefined // false	Compara tipo y valor exacto

Convertir de un tipo a otro

```
let numero3 = 25; // número  
console.log(numero3); // 25  
console.log(typeof numero3); // "number"
```

```
numero3 = String(numero3); // Convertimos a string  
console.log(typeof numero3); // "string"  
console.log(numero3); // "25"
```

typeof dice el tipo de dato (number, string, etc.).

String() es la forma correcta de convertir algo a string.

Después de la conversión, `numero3` ya no es un número, sino el texto "25".

Conversion a booleanos

Númérico a booleanos 0= false, 1> true

Cadena de texto a booleanos " " = false "a" = verdadero

c

Clase: plantillas para crear objetos.

Java Script

DOM: Document Object Model: representación estructurada del contenido HTML de una página, como un árbol. Convierte el HTML en modelo de objetos (nodos), cada etiqueta HTML es un nodo que JavaScript puede leer y modificar en DOM para cambiar el contenido, estilo o estructura.

Puede: leer y modificar contenido, cambiar estilos, agregar y eliminar elementos, capturar eventos-

Ej. `document.getElementById("titulo").innerText = "Hola Erika!";`

Selección de elemnetos

Método	Uso
<code>document.getElementById("id")</code>	Selecciona por ID
<code>getElementsByName()</code>	Colección de elementos por clase (HTMLCollection)
<code>getElementsByTagName()</code>	Colección por nombre de etiqueta (HTMLCollection)
<code>document.querySelector("selector")</code>	Selecciona el primer match (por clase, id o etiqueta)
<code>document.querySelectorAll("selector")</code>	Devuelve todos los elementos que coincidan (NodeList)
<code>element.parentNode</code>	Accede al padre
<code>element.children</code>	Accede a hijos
<code>element.setAttribute("attr", "valor")</code>	Cambia atributos
<code>element.value</code> (en inputs)	Accede o cambia el valor

Ej. `const titulo = document.getElementById("titulo");`
`const boton = document.querySelector(".btn");`

Modificar contenido o estilos

Método	Uso	Ejemplo
<code>.innerText</code>	Texto visible	<code>titulo.innerText = "Nuevo texto";</code>
<code>.innerHTML</code>	Obtiene/cambia HTML interno del elemento	<code>element.innerHTML = "Hola";</code>

Agregar o quitar clases

Método	Ejemplo
<code>.add</code>	<code>element.classList.add("activo")</code>
<code>.remove</code>	<code>element.classList.remove("oculto")</code>
<code>.toggle</code>	<code>element.classList.toggle("modoOscuro")</code>

Nodos

Método	Uso	Ejemplo
.createElement	Crea nuevos elementos	<code>const nuevoParrafo = document.createElement("p"); nuevoParrafo.innerText = "Hola!";</code>
.cloneNode(true)	Clona un nodo (si true, también clona hijos)	<code>original.cloneNode(true)</code>
.classList.add()	Agrega una clase al elemento	<code>div.classList.add("activo")</code>
.appendChild	Añade elementos Inserta un nodo (elemento real del DOM) sin borrar lo anterior.	<code>document.body.appendChild(nuevoParrafo);</code>
.remove	Quita elementos	<code>element.remove(); // moderno y simple</code>

```
ej.
<button id="saludoBtn">Saludar</button>
<p id="mensaje"></p>

<script>
  const btn = document.getElementById("saludoBtn");
  const mensaje = document.getElementById("mensaje");

  btn.addEventListener("click", () => {
    mensaje.innerText = "¡Hola, Erika!";
    mensaje.style.color = "purple";
  });
</script>
```

Insertar elementos

Método	Posición	Ejemplo
.prepend()	Inserta al inicio del contenido (como hijo)	<code>div.prepend(nuevo)</code>
.append()	Inserta al final del contenido (como hijo)	<code>div.append(nuevo)</code>
.before()	Inserta antes del elemento (como hermano)	<code>elemento.before(nuevo)</code>
.after()	Inserta después del elemento (como hermano)	<code>elemento.after(nuevo)</code>

.insertAdjacent

Método	Posición	Uso	Ejemplo
.insertAdjacentElement()	"beforebegin"	antes del elemento actual	el.insertAdjacentElement("afterend", nuevo)
	"afterbegin"	como primer hijo del elemento	
.insertAdjacentHTML()	"beforeend"	como último hijo del elemento	el.insertAdjacentHTML("beforebegin", " Hola ")
	"afterend"	después del elemento actual	

Colecciones y propiedades utiles

Método	Uso	Ejemplo
.length	Cantidad de elementos en una colección	lista.length
.item(i)	Accede al elemento en la posición i de una colección	lista.item(0)

Eventos: Detectan acciones del usuario (click, teclado, etc.) puede ser detectado por JavaScript para ejecutar código.

	Que pasa	Ejemplo
addEventListener()	Decirle al navegador: "cuando pase X evento, corre esta función".	<pre> boton.addEventListener("click", () => { alert("Diste click!"); }); </pre>
event.preventDefault()	Previene el comportamiento por defecto (como enviar un form)	event.preventDefault()
event.target.value	Obtiene el valor del input donde ocurrió el evento	event.target.value.trim().toUpperCase()
.blur()	Quita el foco de un elemento (como cuando sales de un input)	input.blur()
"mouseover" "onmouse"	Se dispara cuando el puntero entra en un elemento.	<pre> <div onmouseover="console.log('Entraste')"> Pasa el mouse por aquí</div> </pre>
"onmouseout"	Se dispara cuando el puntero sale del elemento.	<pre> <div onmouseout="console.log('Saliste')">Pas a el mouse y luego quítalo</div> </pre>

Java Script

"keydown"	Se dispara cuando presionas una tecla (cualquier tecla).	<pre>document.addEventListener("keydown", function (e) { console.log("Tecla presionada:", e.key); });</pre>
"submit"	Se dispara cuando envías un formulario <form> (Enter o botón) de tipo submit o presionar Enter dentro de un input).	<pre><form id="miFormulario"> <input type="text" name="nombre" /> <button type="submit">Enviar</ button> </form> <script> document.getElementById("miFormulario ").addEventListener("submit", function (e) { e.preventDefault(); // Evita que recargue la página console.log("Formulario enviado"); }); </script></pre>

Ej: hacer clic, mover el mouse, escribir en un input, cargar la página, etc.

```
Ej. document.getElementById("bottom").addEventListener("click", function() {
    alert("¡Hiciste clic!");
});
```

Elementos

Método	Tipo	Ejemplo
maxlength	Atributo HTML	Se usa en inputs para limitar el número de caracteres.
uppercase	Transformación de texto	.toUpperCase() a un valor.

Función anónima: sin nombre propio. Se suele usar cuando quieres definir la función *justo en el lugar donde se necesita*, como en eventos.

El evento no ejecuta nada por sí solo.

Necesita una función que le diga qué hacer cuando pase.

Si no pones la función → el evento ocurre pero no hay reacción.

La función anónima es una forma práctica y directa de "qué hacer" en el momento.

Java Script

e

ECMAScript: estándar oficial sobre el que se basa JavaScript. (let, const, funciones flecha (=>), template strings, etc., fueron añadidos en ES6 2015)

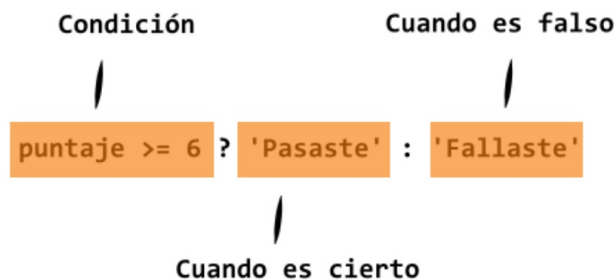
Estructuras de control: permiten dirigir el flujo del programa, es decir, decidir qué se ejecuta y cuándo, dependiendo de condiciones.

Estructura	Función	Ejemplo
Condicionales Toman desiciones	Ejecutan código si se cumple (o no) una condición if / else / else if	<pre>let edad = 18; if (edad >= 18) { console.log("Eres mayor de edad"); } else { console.log("Eres menor de edad"); }</pre>
	Evalúa múltiples posibles valores de una variable switch	<pre>let color = "azul"; switch (color) { case "rojo": console.log("Elegiste rojo"); break; case "azul": console.log("Elegiste azul"); break; default: console.log("Color no reconocido"); }</pre>
Bucles Repeticiones de acciones	for: Se repite un número de veces definido	<pre>for (let i = 0; i < 5; i++) { console.log(i); }</pre>
	while: Se repite mientras se cumpla una condición	<pre>let i = 0; while (i < 5) { console.log(i); i++; }</pre>
	do...while: Se ejecuta al menos una vez, y luego repite si la condición es verdadera	<pre>let i = 0; do { console.log(i); i++; } while (i < 5);</pre>

Java Script

	for...of: Recorre elementos de un array	let frutas = ["manzana", "pera", "uvas"]; for (let fruta of frutas) { console.log(fruta); }
	for...in: Recorre propiedades de un objeto	let persona = { nombre: "Erika", edad: 22 }; for (let clave in persona) { console.log(clave + ": " + persona[clave]); }
Operador ternario	Forma corta de un if/else	let edad = 20; let acceso = edad >= 18 ? "Permitido" : "Denegado";

If ternario



f

Funciones: bloque de código reutilizable que ejecuta una tarea cuando se llama. Se declara, pasa datos (parámetros) y retorna un resultado.

```
Ej. function saludar(nombre) {  
  return "Hola, " + nombre;  
}  
saludar("Erika"); // "Hola, Erika"
```

Flecha (arrow functions): forma más corta de escribir funciones se declaran con palabras reservadas **var, let, const**.

Se escriben: bloque de código reutilizable que ejecuta una tarea cuando se llama. Se declara, pasa datos (parámetros) y retorna un resultado

`const + nombreFunción + operador = + (parámetros) + fat arrow => + {código} +`

Si es una línea única, se pueden omitir `{ }` y **return**.

Ej2. `const sumar = (a, b) => a + b;`

Consideraciones

	Que pasa	Ejemplo
Usar const para declarar la función	Es una buena práctica para evitar que se sobrescriba. Mejora legibilidad y seguridad.	const duplicar = x => x * 2;
Si tiene 1 parámetro,	Puedes omitir los paréntesis ()	const saluda = nombre => `Hola, \${nombre}`; // Paréntesis y return omitidos
Si tiene 1 instrucción (una sola línea)	Puedes omitir {} y return.	
Si usas {} debes escribir return tú mismo	Si abres llaves, el retorno ya no es implícito.	const convertirUnidades = cm => { let metros = cm / 100; return `\${cm} equivalen a \${metros} metro/s`; };

```
Ej console.log
function saluda(nombre) {
  return `Hola, ${nombre}`;
}
```

```
console.log(saluda("Erika")); // imprime en consola: Hola, Erika
```

```
Ej return
function saluda(nombre) {
  console.log(`Hola, ${nombre}`);
}
```

```
let resultado = saluda("Erika"); // imprime en consola: Hola, Erika
console.log(resultado);          // imprime: undefined
```

Diferencias entre console.log y return

	console.log	return
Qué hace	Imprime un valor en la consola (para el usuario/dev)	Devuelve un valor desde una función
Propósito principal	Depurar / ver resultados en tiempo real	Usar el valor fuera de la función / seguir la lógica
Dónde se ve	En la consola del navegador o terminal	No se "ve" directamente, llama la función

Java Script

Afecta el flujo del programa	No	Sí, termina la función y pasa el valor
Puede usarse fuera de una función	Sí	No tiene sentido fuera de una función
Devuelve algo	No (devuelve undefined)	Sí, lo que le digas que devuelva
Ejemplo	<code>console.log("Hola")</code>	<code>return "Hola"</code>

h

Hosting: Elevar, mueve las declaraciones al principio del archivo o bloque al momento de interpretar el código.

Pero solo las declaraciones son elevadas, no las asignaciones.

Las funciones tradicionales se elevan **completas**.

Las variables con **var** se elevan pero su valor será **undefined**.

Las variables con **let** y **const** también se elevan, pero **no puedes usarlas antes** de declararlas (zona temporal muerta, es decir da error).

Las **funciones flecha o anónima** con **const** o **let** No se pueden usar antes de su declaración. Provocan error.

`function getData();` Este tipo de función se "hoístea" (JS la mueve al tope internamente), se puede llamar arriba sin errores.

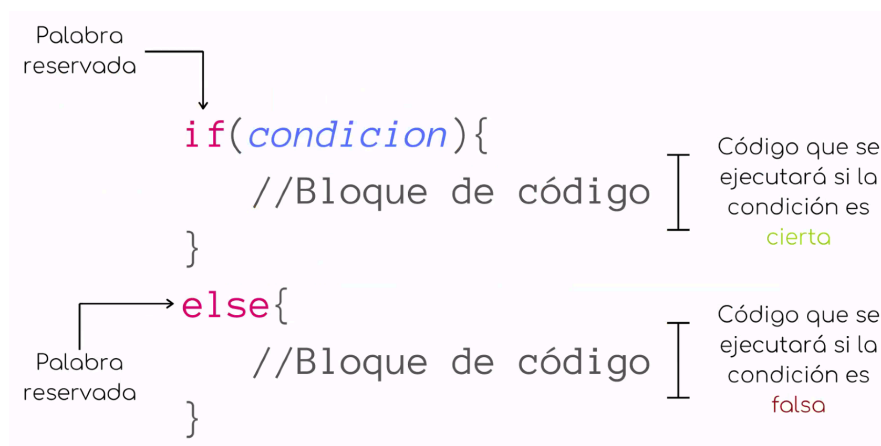
i

If else: condicional si la condición se cumple (if-true)

Dentro del mismo bloque de código

Si la condición se cumple; **if**

Si no cumple; **false** y se ejecuta secuencia dentro de la sentencia; **else**



Java Script

j

JavaScript: lenguaje de programación de alto nivel, scripting multiplataforma y orientado a objetos. Da dinamismo y funcionalidad `<script></script>`.

Lenguaje interpretado, es decir **no necesita compilarse previamente** para ejecutarse, o sea se ejecuta línea por línea por un intérprete, que en este caso es el navegador (como Chrome, Firefox) o el motor como Node.js.

Característica	Java	Java Script
Lenguaje	Compilado	Interprete
Plataforma	Java Virtual Machine (JVM)	Navegadores y Node.js
Sintaxis	Más estricta (fuertemente tipado)	Más flexible (débilmente tipado)
Paradigma	Orientado a objetos	Multiparadigma (OO + funcional)
Casos de uso	Apps empresariales, Android	Web, frontend, backend con Node.js
Ejecución	Necesita compilar .java → .class	Se ejecuta directamente

JavaScript es el lenguaje original, flexible pero con menos control sobre tipos y errores en tiempo de compilación.

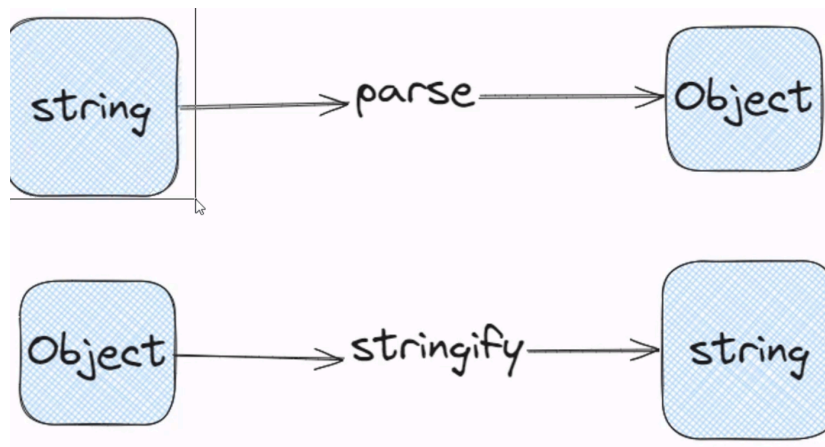
TypeScript es JavaScript + tipos fuertes y características extra para hacer código más seguro y fácil de mantener en proyectos grandes.

JSON – JavaScript Object Notation: Formato estándar basado en texto para representar datos estructurados basados en sintaxis de objetos JavaScript, ligero intercambio de datos, guarda y enviar información entre el frontend (navegador) y el backend (servidor). Es una forma de representar objetos en texto plano.

- Parece un **objeto**, pero es **texto**
- Usa **comillas dobles** " " (**obligatorio**).
- **No acepta comillas simples** ' ' ni propiedades sin comillas.
- Puede contener **objetos, arrays, strings, números, booleanos y null**. Solo puede guardar **datos simples: no puede contener funciones, undefined, ni Date** (las fechas deben guardarse como string).

```
{ "nombre": "Erika",  
  "edad": 22,  
  "esEstudiante": true,  
  "datosExtra": {  
    "ciudad": "CDMX",  
    "lenguajes": ["HTML", "CSS", "JavaScript"]  
  }  
}
```

Java Script



Conversión JSON y JS

Acción	Metodo JS	Explicación
JSON → Objeto JS	JSON.parse(jsonString)	Convierte texto JSON a objeto usable
Objeto JS → JSON (texto)	JSON.stringify(objetoJS)	Convierte un objeto JS a texto JSON válido

JSDoc

Etiqueta	Descripción
@class	Indica que el bloque documenta una clase.
@property {tipo} nombre	Documenta una propiedad de la clase y su tipo.
@static	Marca el método o propiedad como estático (de la clase, no de instancias).
@param {tipo} nombre	Describe un parámetro de función/método con su tipo y nombre.
@returns {tipo}	Describe el tipo que devuelve una función.
@extends ClasePadre	Indica que una clase hereda de otra.
@override	Indica que un método sobrescribe uno de la clase padre.

bash

Entro a la carpeta del proyecto hasta el JS que contiene main.js

```
MacBook-Pro-EA:JS apple$ cat main.js
```

```
MacBook-Pro-EA:JS apple$ jsdoc main.js
```

Aquí al ejecutar jsdoc lee los @ y va a crear la carpeta out/

Después de cada cambio/modificación jsdoc main.js

Jsdoc.app para ver documentación

Java Script

m

Math: objeto incorporado que ofrece funciones y constantes matemáticas. No se instancia (no se usa `new Math()`), simplemente se llama como `Math.función()`.

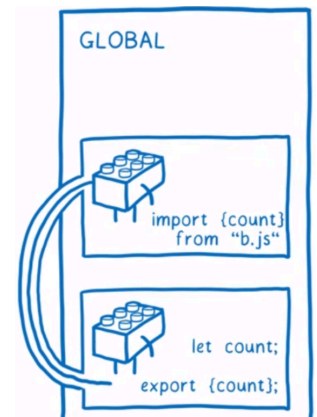
Math	Que hace	Estructura
<code>Math.round(x)</code>	Redondea al entero más cercano	<pre>// Aleatorio entre 0 y 9 (entero) let aleatorio = Math.floor(Math.random() * 10); // Entre 1 y 10 (ambos incluidos) let aleatorio2 = Math.floor(Math.random() * 10) + 1;</pre>
<code>Math.trunc(x)</code>	Elimina decimales, se queda con la parte entera	
<code>Math.max(a, b, ...)</code>	Devuelve el mayor	
<code>Math.min(a, b, ...)</code>	Devuelve el menor	
<code>Math.random()</code>	Número aleatorio entre 0 y <1	

Módulos: mejora organización permitiendo agrupar variables y funciones .
ECMAScript Modules (ES Modules - ESM) – **import/export**
Estándar oficial desde ECMAScript 2015 (ES6).
Compatible con navegadores modernos y Node.js.

```
export function miFuncion() {}
import { miFuncion } from './archivo.js';
```

Para que funcione en el navegador, en **html** agregar:

```
<script type="module" src="main.js"></script>
```



- Los archivos deben tener **extensión .js**
- Usar **import** (asíncrono) en el archivo .js que contiene lo que se quiere compartir (funciones, variables, clases, etc.).

```
export function validarRFC(rfc) {
  const regex = /^[A-Z]{4}[0-9]{6}[A-Z0-9]{3}$/;
  return regex.test(rfc);
}
// Exportación por default (solo una por archivo)
export default function validarCantidad(cantidad) {
  return ....;
}
```

- Usar **export** en el archivo .js donde se van a usar esas funciones exportadas.
- Usar rutas relativas (./archivo.js)
// Importar funciones nombradas (con llaves)
import { validarEmail, validarRFC } from './validaciones.js';

Java Script

// Importar función por defecto (no se puede juntar como las demás, sin llaves, puedes renombrarla)

import validarCantidad from './validaciones.js';

- Cada archivo es un módulo con su propio scope.

CommonJS Modules (CJS) — require/module.exports (exclusivo de Node.js)

// En CommonJS

module.exports = function () { ... };

const miFuncion = require('./archivo');

n

Numéricos: numéricos enteros y reales

Inicia operaciones de izquierda derecha

Suma: +

Resta: -

Multiplicación: *

Cociente: /

Residuo: %

Potencia **

Null: valores nulos o vacíos.

o

Objeto: instancia, materialización de una clase.

Propiedades: información (datos) que almacena el objeto

Métodos: acciones que un objeto puede ejecutar (con las propiedades).

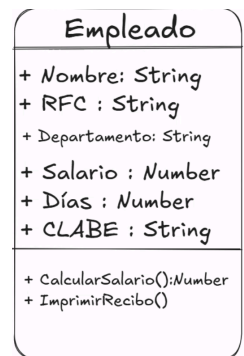
Object: estructura que permite almacenar múltiples datos relacionados como pares clave: valor. Se usa para representar entidades complejas y agrupar información.

Cada dato dentro se compone de una clave (key) y un valor (value).

Se accede a los valores usando punto . o corchetes []:

```
let persona = {  
  nombre: "Erika",  
  edad: 22,  
  estudia: true  
};
```

```
console.log(persona.nombre); // Erika  
console.log(persona["edad"]); // 22
```



Java Script

Operadores: símbolos que le dicen al programa que realice una operación entre uno o más valores.

Tipo	Operadores	Descripción	
Aritmético	+		$2 + 3 = 5$ o "a" + "b"
	-		
	*		
	/		
	%		
	**		

Tipo	Operadores	Descripción	
Asignación	=	Asignación	$x += 5 \rightarrow x = x + 5$
	+=		
	-=		
	*=		
	/=		

Java Script

Tipo	Operadores	Descripción	Comparación	Return
Comparación	= equal to	Igualdad débil	X == 8 X == "5"	False True
	=== equal value and equal type	Igualdad estricta	X === 5 X === "5"	True False
	!= not equal	Diferente débil	X != 8	True
	!== not equal value or not equal type	Diferente estricto	X !== 5 X !== "5" X !== 8	False True True
	> greater than	Mayor que	X > 8	False
	< less than	Menor que	X < 8	true
	>= greater than or equal to	Mayor o igual	X >= 8	False
	<= less than or equal to	Menor o igual	X <= 8	true
Tipo	Operadores	Descripción		
Lógicos	&&	AND (y)	true && false → false	
		OR	true false → true	
	!	NOT (no)	!true → false	
Tipo	Operadores	Descripción		
Ternario		Forma compacta de if/else	let edad = 20; let acceso = edad >= 18 ? "Permitido" : "Denegado";	
Tipo	typeof	Tipo de dato	typeof 5 → "number"	
	instanceof	Verifica si es instancia de una clase	x instanceof Array	

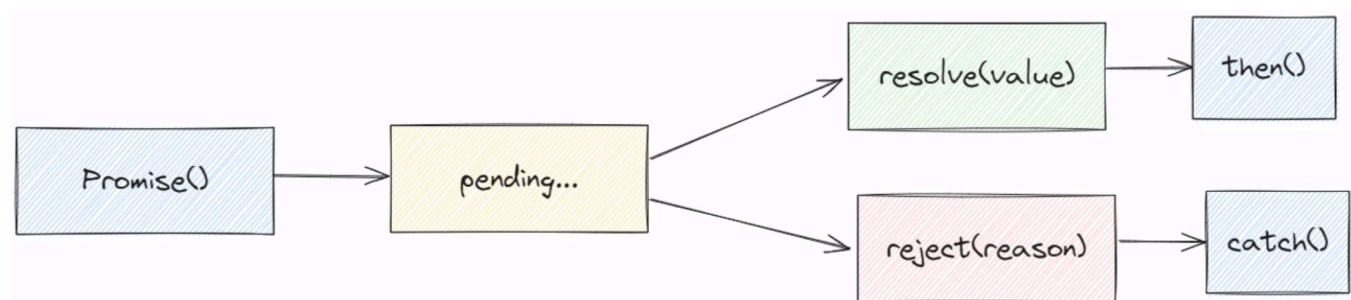
Java Script

P

Promesas: objeto que representa un valor que aún no está disponible pero lo estará en el futuro, después de que se complete una operación asíncrona. (Llama APIs)

Estados de una promesa

pending	en proceso, aún sin resolver
fulfilled .then()	resuelta exitosamente
rejected .catch()	rechazada (hubo un error)



Sintaxis

```
const promesa = new Promise((resolve, reject) => {  
  // proceso asíncrono  
  if (todoBien) {  
    resolve("Todo salió bien");  
  } else {  
    reject("Algo salió mal");  
  }  
});
```

.then() y .catch()

```
promesa  
  .then((resultado) => {  
    console.log("Éxito:", resultado);  
  })  
  .catch((error) => {  
    console.error("Error: ", error);  
  });
```

setTimeout

```
const tarea = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    const completado = true;  
    if (completado) {
```

Java Script

```
        resolve("Tarea completada");
    } else {
        reject("La tarea falló");
    }
}, 2000);
});
```

```
tarea
    .then((msg) => console.log(msg))
    .catch((err) => console.log(err));
```

Buena practica para timeout, definirla como constante y mencionarla en promesas como

```
vscode sugiere
setTimeout(() => {
    }, timeout);
```

fetch

```
fetch("https://fakestoreapi.com/products/1")
    .then((response) => response.json())
    .then((data) => console.log(data))
    .catch((err) => console.log("Error: ", err));
```

Pruebas unitarias (unit testing): verifica que una función específica ("unidad" de código) hace lo que debe hacer.

Usando note se puede utilizar frameworks js: jest, unit.js, mocha, karma, Qunit, Jasmine, AVA, chai, vitest, playwright/cypress, etc..

Jest: npm — save — devtest

En el archivo operaciones.test.js

```
test("mult 9 + 9 igual a 81", ()=>{
    expect(oper.mult(9,9)).toBe(81);
});
```

En el archivo Operaciones.js

```
function mult(a,b){
    return a*b;
}
```

Ejecutar npm run test en bash

s

Scope: Alcance, define desde dónde se puede acceder a una variable o función.

Global: variables declaradas fuera de funciones o bloques están disponibles en todo el script.

```
Ej. let nombre = "Erika"; // global
function saludar() {
    console.log(nombre); // puede acceder
```

Java Script

```
}
```

Local: declaradas dentro de funciones o bloques solo existen ahí.

```
Ej. function ejemplo() {  
  let mensaje = "Hola"; // local  
}
```

```
console.log(mensaje); // ❌ Error: no está definida afuera
```

***let** y **const** respetan el **bloque** en el que se declaran.

var no respeta el bloque, solo la **función**.

```
{  
  let x = 10;  
}  
  
console.log(x); // ❌ Error
```

```
{  
  var y = 10;  
}  
  
console.log(y); // ✅ 10
```

Storage; web storage. Proporciona mecanismos mediante los cuales el navegador almacena información de tipo clave/valor

Local Storage	Guarda datos en el navegador sin fecha de expiración . Permanece incluso si cierras y vuelves a abrir el navegador. Solo se borra si el usuario lo elimina manualmente o con código.	<pre>localStorage.setItem("clave", "valor"); const dato = localStorage.getItem("clave"); localStorage.removeItem("clave"); localStorage.clear(); // Borra todo</pre>
Session Storage	Guarda datos solo durante la sesión actual del navegador. Se borra automáticamente al cerrar la pestaña o navegador.	<pre>sessionStorage.setItem("clave", "valor"); const dato = sessionStorage.getItem("clave"); sessionStorage.removeItem("clave"); sessionStorage.clear();</pre>

String: cadenas de texto, combinación de carácter alfanuméricos y caracteres especiales, se escriben dentro de comillas ""

t

Typeof: dice el tipo de dato (number, string, etc.).

Type	Typeof
<code>typeof "Hola"</code>	<code>// "string"</code>

Java Script

typeof 10	// "number"
typeof true	// "boolean"
typeof []	// "object" (arrays)
typeof {}	// "object"
typeof null	// "object" ! bug histórico
typeof function(){} 	// "function"

tabindex="0": es parte de cómo JS interactúa con HTML, atributo HTML no hace nada por si solo (onkeydown), permite que un elemento se pueda enfocar con la tecla Tab.

Valor "0": incluir este elemento en el orden natural de navegación con Tab.

Se usa en elementos que no son interactivos por defecto (<div>, , etc.), para ser accesibles.

tabindex="-1" → El elemento puede recibir foco solo con JavaScript, no con Tab.

tabindex="1" o mayor → Cambia el orden natural del foco, pero no se recomienda (mala práctica de accesibilidad).

OnClick: Evento que ejecuta una función cuando el usuario hace clic con el **mouse/touch**.

Onkeydown: Evento que se dispara al presionar una tecla mientras el foco está en el elemento, lo hace funcionar Enter o Espacio, se usa junto con **tabindex="0"** para que el elemento también funcione con **teclado** (por accesibilidad).

Estructura uso de eventos

		Ejemplo
tabindex="0"	para que se pueda navegar con Tab.	
onkeydown	Detecta teclas presionadas como Enter o espaciadora	<code>element.onkeydown = (e) => { console.log(e); }</code>
onclick	Ejecuta código al hacer clics normales con el mouse.	<code>element.onclick = () => { alert("Clic"); }</code>
role="button"	para que los lectores de pantalla lo identifiquen como botón.	
addEventListener	Forma moderna y flexible de manejar eventos	<code>element.addEventListener("click", handler);</code>

u

Undefined: Variable indefinida

Palabra reservada + nombre de la variable

Java Script

v

Variables: donde se guardan los datos, contenedores esenciales para almacenamiento y manipulación de datos.

Se declaran usando palabras reservadas como **var**, **let** y **const**.

También existen variables **indefinidas**.

Palabra reservada + nombre de la variable + operador = + asignación

Ej. Let nombre = "Juan";

Variable	Cuando usar	Scope	Redeclarar	Ejemplo
let	si la variable puede cambiar	Bloque {}	No	var edad = 22; let nombre = "Erika"; const PI = 3.14; edad = 23; //  nombre = "Rodrigo"; //  PI = 3.1415; //  Error
var	también se puede cambiar, ya casi no se usa	Funcional	Si	
const	no cambia	Bloque {}	No	

Validaciones Regex (Expresiones regulares)

Patrones que se usan para buscar o validar texto con una estructura específica, como correos, contraseñas, RFC, etc.

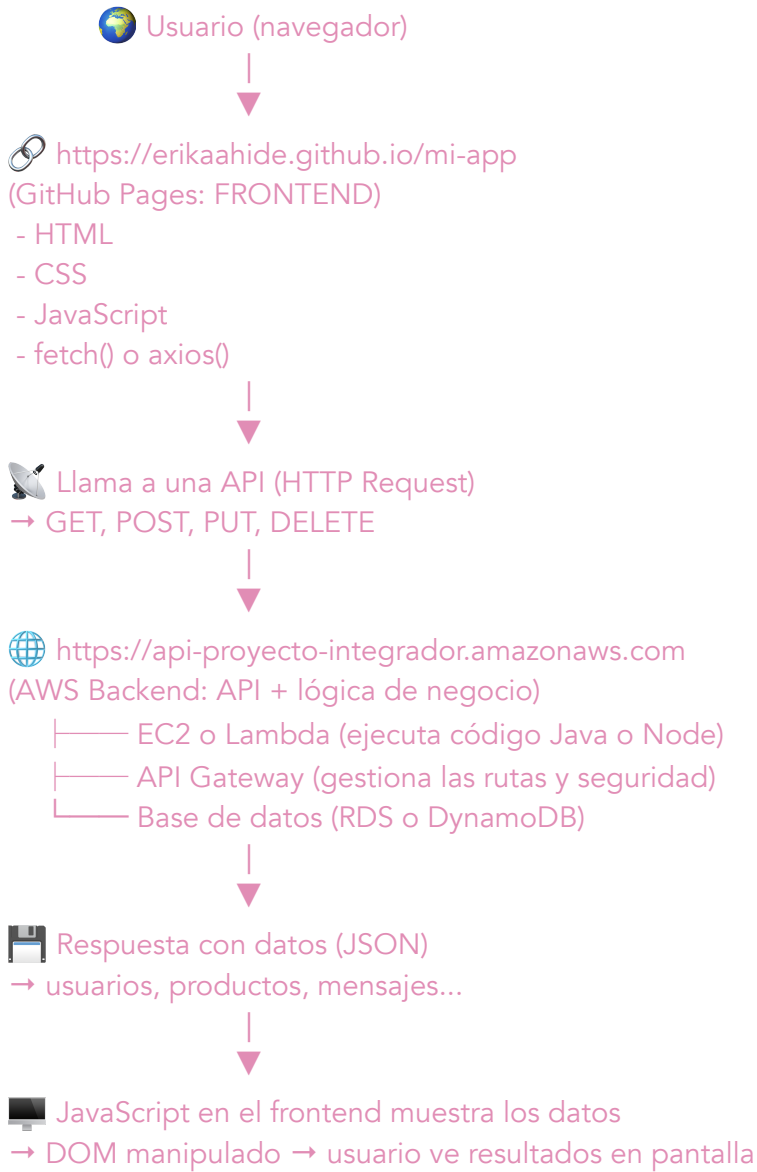
Verifica que un texto cumpla cierto patrón

En validaciones de entrada de datos (formularios, APIs, etc.)

Java Script

x

Arquitectura web full stack



Buenas prácticas

Asignar nombres significativos para dar valor y entender variables

Utilizar convención **camelCase** para nombrar **variantes**

Utilizar convención **UPPER_SNAKE_CASE** para nombrar **constantes**

Utilizar nombres cortos máximo **3** palabras

Declarar todas las variantes al inicio de código